

TRANSMISIÓN DE UNA IMAGEN UTILIZANDO USRPs Y SIMULINK/MATLAB

Documentación Fase 1 — Fase I. Sistema de transmisión/recepción.

Laboratorio II de Telecomunicaciones

Universidad del Cauca — Facultad de Ingeniería Electrónica y Telecomunicaciones

Tabla 1. Integrantes del Equipo de desarrollo.

Integrante	Rol
Luis Miguel Gomez Muñoz	Desarrollador
Jose Martin Gonzalez Joaqui	Desarrollador
Briyith Vaneza Guacas Narvaez	Desarrolladora
Karol Tatiana Palechor Valencia	Desarrolladora

1. Descripción de la Fase 1.

Esta fase tiene por objetivo construir un canal de comunicación empleando USRPs y archivos de Simulink, de MatLab.

1.1. Etapas del sistema.

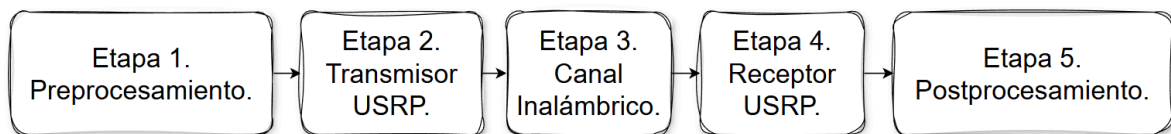


Imagen 1. Etapas del sistema.

La imagen 1 representa las etapas relevantes del sistema que permiten comprender cómo se procesa la señal, trabajando de la siguiente manera:

- Etapa 1. Preprocesamiento: Esta etapa de la preparación de la imagen realiza el proceso necesario para convertir la imagen escogida en bits, para continuar sin problemas a la siguiente etapa. La imagen se comprime reduciendo su paleta a 256 colores, codificando cada píxel en 8 bits en lugar de 24. Los píxeles se convierten a una secuencia binaria y se dividen en bloques de tamaño fijo. Cada bloque recibe un número de secuencia en su encabezado para permitir el reordenamiento en recepción.
- Etapa 2. Transmisor USRP: Luego de obtener los bits correspondientes a la imagen, sirven como entrada de la presente etapa, encargándose de convertir estos bits en una señal transmisible por el dispositivo hardware USRP. Los bits se encapsulan en tramas con un preámbulo de referencia y se aleatorizan para balancear ceros y unos. Se modulan en QPSK -4 estados de fase, 2 bits por símbolo- y se filtran para limitar el ancho de banda. El USRP transmite a 915 MHz a 2.5 Msym/s.
- Etapa 3. Canal inalámbrico: Esta etapa destaca por las consecuencias que tiene el paso de la señal por la presente, causando cambios que se busca sean controlados en la siguiente etapa,

pero notable en los resultados. La señal se degrada por ruido, diferencia de frecuencia entre osciladores y desfase de portadora.

- Etapa 4. Receptor USRP: Esta etapa recibe la señal degradada de la etapa anterior, con el objetivo de recuperar la señal original por métodos específicos. El receptor normaliza la potencia, corrige el offset de frecuencia en dos etapas (gruesa y fina) y recupera el instante óptimo de muestreo mediante lazos de control. Luego detecta el preámbulo para localizar el inicio de cada trama, descrambliza (proceso inverso a scramble) y demodula los símbolos a bits.
- Etapa 5. Postprocesamiento: Con la señal recibida de la etapa anterior, se realiza la reconstrucción de la imagen, buscando convertir los bits resultantes en la imagen esperada. Las tramas se reordenan por número de secuencia. Se calcula la BER comparando bits transmitidos y recibidos. Eliminados los encabezados, los bits se convierten de vuelta a píxeles indexados y se restaura la imagen en color usando la paleta guardada por el transmisor.

2. Implementación.

2.1. Etapa 1. Preprocesamiento.

La etapa de preprocesamiento consiste en realizar la preparación de la imagen para la respectiva conversión de los datos en el formato requerido para la siguiente etapa. Esto permite que la siguiente etapa procese los datos correctamente y se envíen por el canal inalámbrico sin inconvenientes.

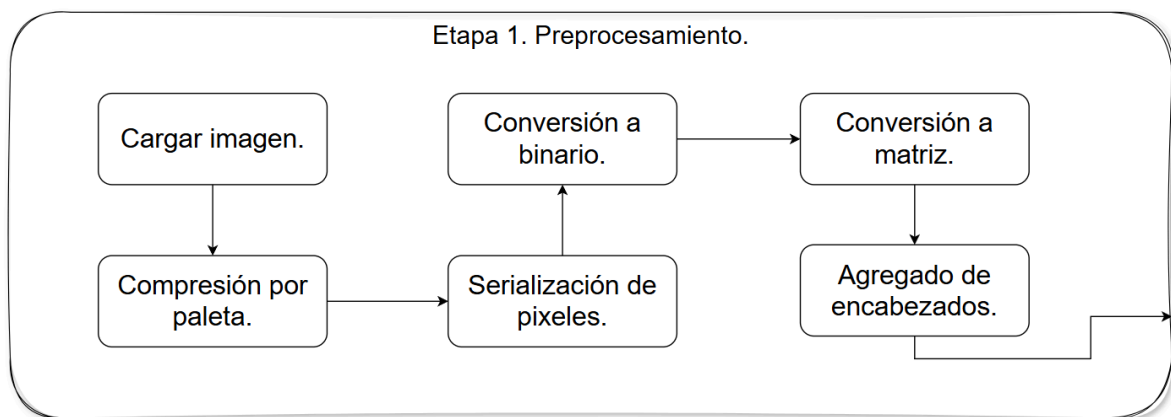


Imagen 2. Proceso de la etapa 1, preprocesamiento.

La imagen 2 proporciona una vista general de los módulos destacados que permite convertir la imagen en el formato adecuado para la siguiente etapa. Estos módulos son funciones de MatLab, detallados de la siguiente manera:

- Cargar imagen: Se encarga de tomar la imagen de un directorio específico y hacerla accesible al entorno de trabajo.
- Compresión por paleta: Esta se encarga de aplicar la compresión por paleta, la cual es una técnica de compresión de imágenes que reduce el tamaño de la imagen al limitar el número total de colores únicos a un máximo de 256, de los millones de colores posibles, y almacenar las referencias a esos colores en una tabla matemática llamada "paleta" o "mapa de colores". Entrega la imagen indexada y la "paleta" en un archivo ".mat".

- Serialización de píxeles: Se encarga de tomar esta imagen indexada, la cual es una matriz dentro del entorno de trabajo, y lo convierte en un vector de una dimensión.
- Conversión a binario: Con el vector obtenido, se procede a convertir cada valor de la imagen indexada en 8 bits.
- Conversión a matriz: Con el vector convertido a bits, se reconstruye este vector en una matriz de 11.190 columnas por trama. Este paso es relevante porque permite enviar la información en tramas, lo cual disminuye significativamente el costo computacional que requiere procesar la imagen en Simulink/MatLab.
- Agregado de encabezados: Para cada trama, se agrega una cabecera que indica el número de la trama en secuencia, lo que permitirá reordenar las tramas en el receptor en caso de recibirlas en desorden.

2.2. Etapa 2. Transmisor USRP.

La etapa de Transmisor consiste en construir los bloques necesarios para tomar la imagen y procesar la misma a través de una cadena de bloques hasta convertirla en una señal de radiofrecuencia lista para ser emitida por el USRP.

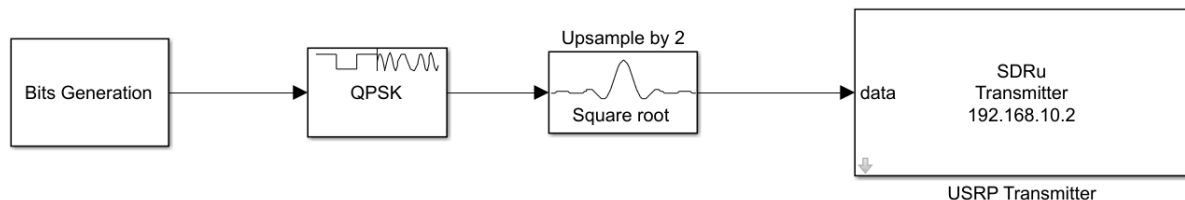


Imagen 3. Transmisor en Simulink/MatLab.

- Bits Generation: Este bloque es el subsistema inicial del transmisor. Se encarga de construir cada trama que será enviada al aire, combinando un preámbulo de sincronización con los bits de imagen, y entregando al resto de la cadena una secuencia binaria estructurada y lista para modular.

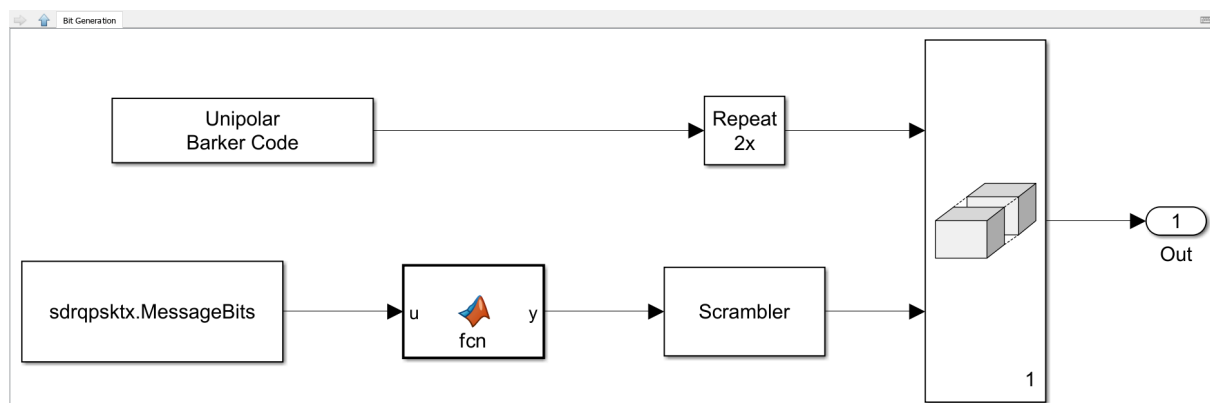


Imagen 4. Subsistema Bits Generation.

- Unipolar Barker Code: Genera la secuencia de 13 bits de Barker en formato unipolar (0s y 1s) que servirá como firma de sincronización al inicio de cada trama.
- Repeat 2x: Duplica la secuencia Barker, resultando en 26 bits de preámbulo. Esto mejora la robustez de la detección en el receptor al tener más bits de referencia para correlacionar.

- `sdrqpsktx.MessageBits`: Es la fuente de datos. Contiene la matriz binaria de la imagen cargada desde el workspace, que constituye el payload de cada trama.
- Bloque MATLAB Function (fcn): Procesa los bits de imagen para adecuarlos al formato de trama esperado por el resto del subsistema, columna por columna de la matriz binaria.
- Scrambler: Aleatoriza los bits del payload para garantizar una distribución balanceada de ceros y unos, necesaria para el correcto funcionamiento de los lazos de sincronización en el receptor.
- Frame Builder (bloque de concatenación): Une el preámbulo Barker duplicado con el payload scramblerizado en una única secuencia continua, formando la trama completa que sale por el puerto Out hacia el modulador QPSK.
- QPSK Modulator Baseband: Este bloque agrupa los bits de dos en dos y asigna a cada par uno de cuatro estados de fase posibles: 0° , 90° , 180° o 270° . El mapeo sigue el código Gray, lo que significa que símbolos adyacentes en la constelación difieren en un solo bit, minimizando los errores cuando el receptor confunde un símbolo con uno vecino.
- Raised Cosine Transmit Filter (Filtro de coseno alzado en transmisión): Da forma al pulso de cada símbolo. El factor de caída (roll-off $\alpha = 0.5$) controla el compromiso entre ancho de banda ocupado e interferencia entre símbolos. El filtro tiene un span de 10 símbolos e introduce un factor de interpolación de 2, duplicando la tasa de muestras de salida.
- USRP Transmitter: Es la interfaz entre Simulink y el hardware. Recibe las muestras en banda base (complejas) y las envía al USRP, que se encarga de trasladarlas a la frecuencia portadora de 915 MHz. La ganancia configurada en 25 dB determina la potencia de transmisión. La tasa de símbolo es 2.5 Msym/s, lo que con el factor de interpolación resulta en una tasa de muestreo de 5 Msps hacia el USRP.

2.3. Etapa 3. Canal inalámbrico.

Esta etapa se caracteriza por realizar las conexiones hardware necesarias para que el enlace de comunicaciones funcione. Los componentes usados para esta etapa son:

Tabla 2. Componentes hardware de la etapa 3.

Componente.	Descripción.	Imagen.
USRP B210.	Es una tarjeta de Radio Definida por Software (SDR) que se conecta a una computadora para enviar y recibir señales inalámbricas en un rango enorme de frecuencias. Funciona como una radio universal reprogramable.	
Antena de 900 MHz en color negro.	Es un componente de comunicación inalámbrica diseñada para operar en el rango de frecuencia de los 800 a 960 MHz. Se presenta en formato de tipo dipolo y la ganancia suele variar entre los 2 dBi y 5 dBi.	

Computadores.	Computadores con MatLab instalado en su última versión con los archivos correspondientes de transmisión y recepción.	
---------------	--	--

La interconexión de estos componentes permiten establecer la etapa 3, canal inalámbrico.

2.4. Etapa 4. Receptor USRP.

Esta etapa se caracteriza por

QPSK Receiver with USRP(TM) Hardware

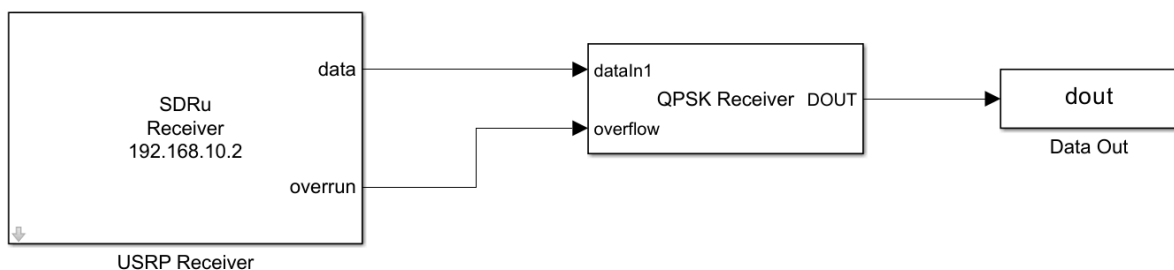


Imagen 5. Receptor en Simulink/MatLab.

- USRP Receiver: Es la interfaz con el hardware USRP. Captura las muestras de la señal recibida a 915 MHz y las entrega en banda base a Simulink. La señal overrun indica si hubo pérdida de muestras por saturación del buffer
- QPSK Receiver: Es el subsistema que contiene toda la cadena de procesamiento digital para recuperar los bits.

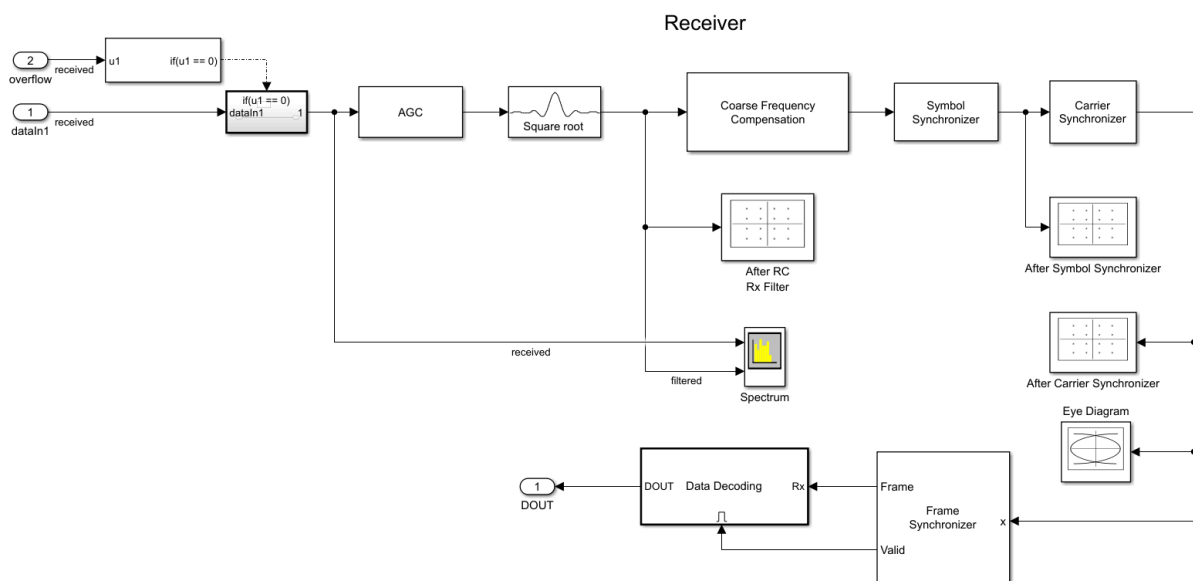


Imagen 6. Subsistema QPSK Receiver.

- Bloque If: Evalúa la señal de “overflow” recibida del USRP. Si su valor es cero (no hubo desbordamiento), activa el subsistema de acción para continuar el procesamiento normal.
- If Action Subsystem: Solo se ejecuta cuando el bloque If lo habilita, es decir, cuando no hay overflow. Su función es dejar pasar las muestras de “dataIn1” hacia la cadena de procesamiento. Si hubo desbordamiento, este subsistema no se activa y las muestras corruptas no continúan.
- AGC (Control Automático de Ganancia): Normaliza la amplitud de la señal recibida a un nivel estable y conocido, independientemente de la distancia o las condiciones del canal.
- Raised Cosine Receive Filter (Filtro de coseno alzado en recepción): Completa el filtrado de pulsos iniciado en el transmisor. Juntos forman un filtro adaptado que maximiza la relación señal-ruido en el instante de muestreo.
- Coarse Frequency Compensation: Estima y corrige el offset de frecuencia grueso entre los osciladores del transmisor y el receptor, usando un algoritmo basado en FFT. Sin esta corrección, la constelación QPSK giraría continuamente.
- Timing Recovery (Symbol Synchronizer): Determina el instante óptimo dentro de cada período de símbolo para tomar la decisión de detección, usando el algoritmo de Gardner. Compensa los retardos de tiempo introducidos por el canal.
- Fine Frequency Compensation (Carrier Synchronizer): Corrige el offset de frecuencia residual y el desfase de portadora que quedaron tras la compensación gruesa, mediante un lazo de seguimiento de fase (PLL).
- Frame Synchronizer: Subsistema que busca el preámbulo Barker en la secuencia de símbolos para identificar el inicio exacto de cada trama. Entrega tramas alineadas de tamaño fijo y una señal “Valid” que indica cuándo la trama detectada es confiable.
- Data Decoding: Es un subsistema que resuelve la ambigüedad de fase del PLL, demodula los símbolos QPSK a pares de bits y aplica el descrambler inverso para recuperar los bits de imagen originales.

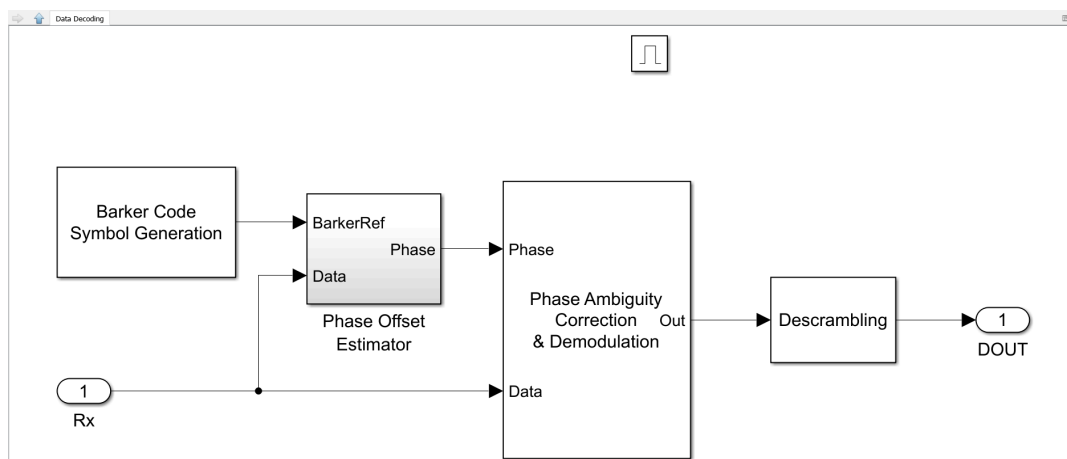


Imagen 7. Subsistema Data Decoding.

- Barker Code Symbol Generation: Genera la versión en símbolos QPSK del preámbulo Barker, es decir, la misma secuencia de referencia que usó el transmisor pero ya modulada. Se usa como patrón conocido para estimar el desfase de fase.
- Phase Offset Estimator: Compara los símbolos recibidos del preámbulo con la referencia Barker generada localmente. De esa comparación calcula cuántos grados de desfase introdujo el canal, entregando un valor de fase estimado.
- Phase Ambiguity Correction & Demodulation: Recibe los datos y el desfase estimado, corrige la rotación de fase residual que el PLL no pudo resolver

(ambigüedad de 90°, 180° o 270°) y demodula los símbolos QPSK a pares de bits.

Phase Ambiguity Resolution
and Demodulation

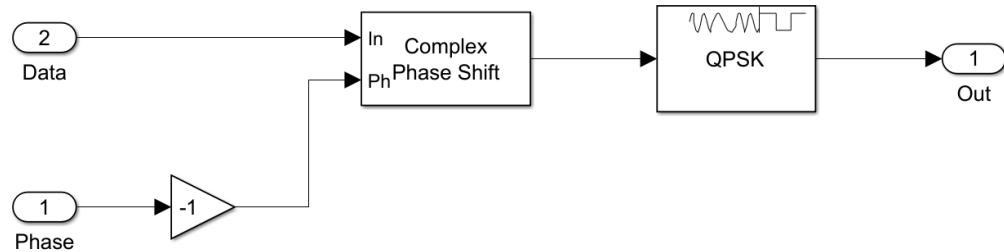


Imagen 8. Subsistema Phase Ambiguity Correction & Demodulation.

- Descrambling: Aplica la misma secuencia pseudoaleatoria del transmisor en sentido inverso para revertir el scrambling y recuperar los bits de imagen originales.
- Data Out: Bloque “To Workspace” que sirve como puerto de salida con los bits recuperados hacia la etapa de postprocesamiento.

2.5. Etapa 5. Postprocesamiento.

Esta etapa de Postprocesamiento se encarga de tomar los bits entregados por la etapa anterior y convertirlos en la imagen resultante, construída mediante los siguiente módulos principales.

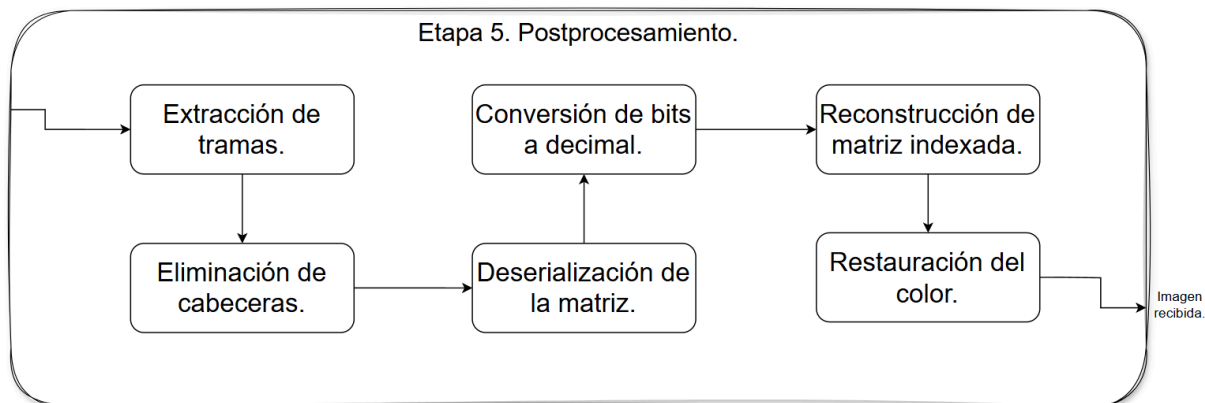


Imagen 9. Etapa 5, postprocesamiento.

- Extracción de tramas: A partir de los bits recibidos, se identifican y reconstruyen las tramas que componen la imagen usando el número de secuencia de cada cabecera. Las tramas faltantes se rellenan con ceros.
- Eliminación de cabeceras: Se remueven los 10 bits de encabezado de cada trama, dejando únicamente los bits que corresponden a datos de imagen.
- Deserialización de la matriz: Los bits se reorganizan de vuelta a la estructura de matriz que tenían antes de ser serializados para la transmisión.
- Conversión de bits a decimal: La secuencia binaria se convierte a valores enteros de 8 bits, recuperando los índices de color de cada píxel.
- Reconstrucción de la matriz indexada: Los valores decimales se reorganizan en la matriz 2D de dimensiones originales, obteniendo la imagen en formato indexado.

- Restauración del color: Usando la paleta de 256 colores guardada por el transmisor, se convierte la imagen indexada de vuelta a RGB, recuperando los colores originales.
- Comparación visual: Se muestra la imagen transmitida y la recibida lado a lado para evaluar cualitativamente el impacto del canal sobre la calidad visual.

2.6. Cálculo de BER.

El preprocesamiento de la señal en ambos computadores de transmisión y recepción permitió obtener la matriz original y compararla con la recibida en recepción, como se especifica a continuación:

- Entrada: Se toman las tramas recuperadas por el receptor y la matriz binaria original generada por el transmisor, ambas con la misma estructura y dimensiones.
- Comparación: Se aplica una operación XOR bit a bit entre ambas matrices. Cada posición donde el resultado es 1 representa un bit que llegó errado.
- Cálculo: El número total de unos resultantes del XOR se divide entre el número total de bits comparados, obteniendo la proporción de bits erróneos sobre el total transmitido.
- Salida: Un valor entre 0 y 1, donde 0 indica que no hubo errores y valores cercanos a 1 indican una transmisión muy degradada.

3. Cambios realizados.

Los cambios realizados respecto a la planeación inicial se enfocan en los siguientes aspectos:

- Codificación y decodificación de canal (Viterbi): El plan incluía un codificador convolucional en el transmisor y un decodificador Viterbi en el receptor para control de errores. En este proyecto, estos no están presentes; el sistema transmite los bits sin codificación de canal, lo que significa que los errores introducidos por el canal no tienen ningún mecanismo de corrección.
- Scrambler: Este proyecto usa un scrambler/descrambler para balancear la secuencia de bits. Este no aparece mencionado en las especificaciones de la versión inicial, es un elemento que se agregó durante el desarrollo.

4. Entregables del Sprint 3.

Tabla 4. Entregables de Sprint 2.

Entregable	Estado
Repositorio de GitHub	Completado

5. Control de Versiones

Los archivos finales fueron subidos al repositorio GitHub, en la rama *dev*, bajo la siguiente estructura:

```
None
/
├── docs/
│   ├── Sprint2_Documentation.pdf
│   ├── Sprint3_Documentation.pdf
│   └── Sprint4_Documentation.pdf
```



```
|   └─ Final_Documentation.pdf
├─ src/
│   └─ ImageLoader/
│       ├── adaptar_tamano.m
│       ├── anadir_cabecera.m
│       ├── cargar_imagen.m
│       ├── comprimir_paleta.m
│       ├── convertir_binario.m
│       ├── main.m
│       └─ serializar_pixeles.m
│   └─ ImageRecovery/
│       ├── calcular_ber.m
│       ├── comparar_imagenes.m
│       ├── convertir_decimal.m
│       ├── descompress_paleta.m
│       ├── deserializar_matriz.m
│       ├── deserializar_pixeles.m
│       ├── eliminar_cabecera.m
│       ├── extraer_tramas.m
│       └─ main.m
│   └─ Receiver/
│       ├── sdrupskrx.slx
│       └─ sdruQPSKReceiverSLInit.m
│   └─ Transmitter/
│       ├── sdrupsktx.slx
│       └─ sdruQPSKTransmitterSLInit.m
│   └─ utils/
│       └─ paleta_colores.mat
├─ data/
│   ├── input/
│   │   └─ italy.jpg
│   └─ output/
│       └─ data_recv.mat
├─ tests/
├─ .gitignore
├─ setup.m
└─ README.md
```

Repositorio: <https://github.com/LMiguelGo/usrp-image-transfer-matlab>